‘’’

LP-III Assignment 5 (Group A - DAA)

Design n-Queens matrix having first Queen placed. Use backtracking to place remaining Queens to generate the final n-queen’s matrix.

Name: Tanmay Sandip Bothara

Roll No: 23 Batch: BE_Comp_A1

‘’’

```
def solveNQueens(n, first_queen_col):
    col = set()
    posDiag = set()
    negDiag = set()

    res = []
    board = [["."] * n for _ in range(n)]

    def backtrack(r):
        if r == n:
            res.append(["".join(row) for row in board])
            return

        for c in range(n):
            if c in col or (r + c) in posDiag or (r - c) in negDiag:
                continue

            col.add(c)
            posDiag.add(r + c)
            negDiag.add(r - c)
            board[r][c] = "Q"

            backtrack(r + 1)
```

```
            col.remove(c)
            posDiag.remove(r + c)
            negDiag.remove(r - c)
            board[r][c] = "."

    col.add(first_queen_col)
    posDiag.add(first_queen_col)
    negDiag.add(-first_queen_col)
    board[0][first_queen_col] = "Q"

    backtrack(1)
    return res


if __name__ == "__main__":
    n = int(input("Enter the size of the chessboard: "))
    first_queen_col = int(
        input("Enter the column for the first queen (0 to n-1): ")
    )

    if n <= 0:
        print("Board size must be positive.")
    elif first_queen_col < 0 or first_queen_col >= n:
        print("Invalid column number.")
    else:
        solutions = solveNQueens(n, first_queen_col)

        if solutions:
            print("One valid solution:")
            for row in solutions[0]:
```

```
            print(" ".join(row))
    else:
        print("No solution exists for the given inputs.")
```

‘’’

OUTPUT:-

Enter the size of the chessboard: 4

Enter the column for the first queen (0 to n-1): 1

One valid solution:

. Q . .

. . . Q

Q . . .

. . Q .

‘’’